

ChainPilot

Agentic AI Supply Chain Control Tower — Architecture Overview

Version v2 · Auto-generated August 21, 2026 · reflects the repository as of this run

1. What is ChainPilot?

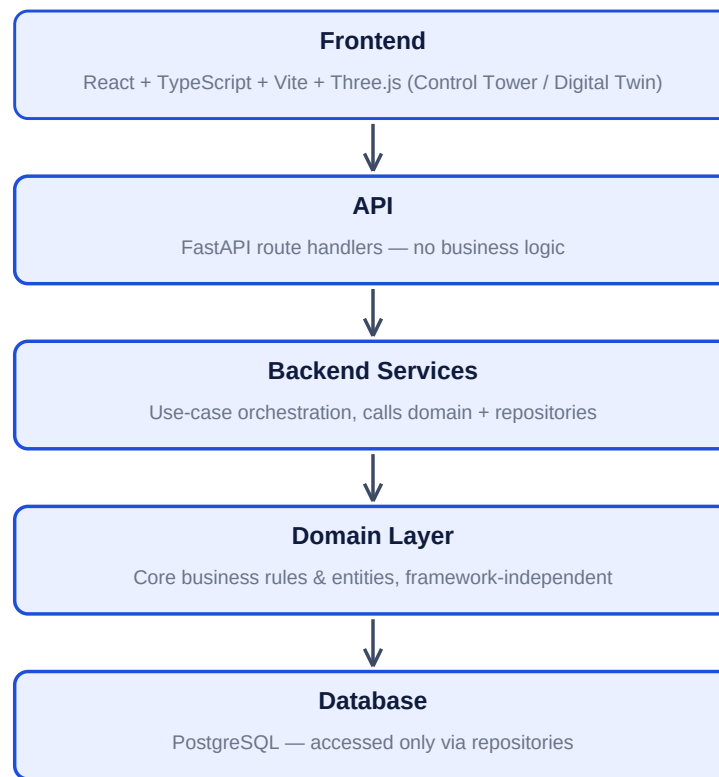
ChainPilot is a platform for operating a supply chain through a live 3D digital twin combined with a network of specialized AI agents. Operators will be able to see warehouses, trucks, docks, and inventory in real time; surface operational exceptions as they emerge; simulate recovery options; and — with human approval — let agents take action.

Design goals

- A believable, real-time 3D digital twin of warehouse and logistics operations (parking lots, dock doors, aisles, bays, trucks, trailers, pallets, shipment routes).
- A modular agent architecture (Orchestrator → specialized agents) exposed through MCP tools, so agents reason over live operational data.
- Human-in-the-loop control: agents recommend and simulate; humans approve; actions execute and are logged as operational events.
- A clean separation between frontend, backend, agents, and MCP so each layer can evolve independently.

2. Application Layers

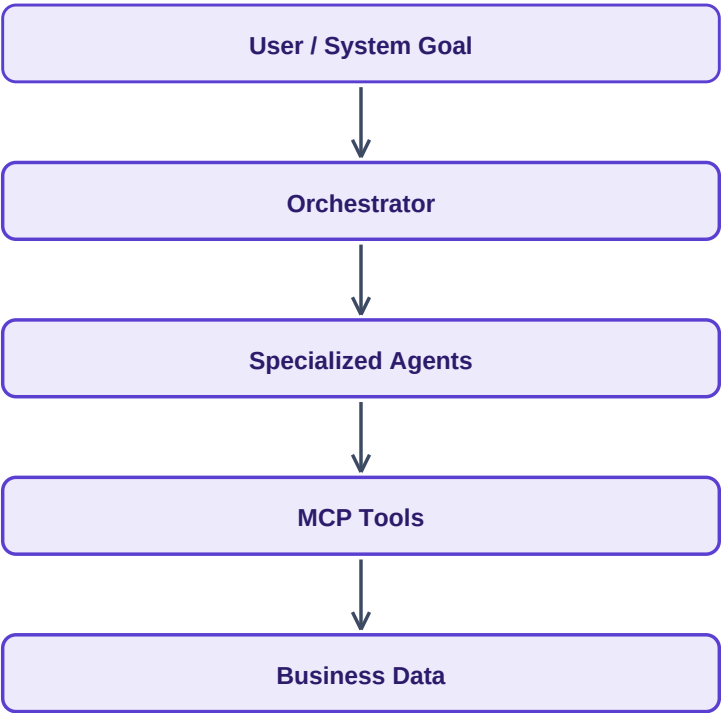
The product stack is a straightforward top-to-bottom layering. Each layer talks only to the one directly below it, which keeps the UI, HTTP concerns, application logic, business rules, and persistence independently replaceable.



Frontend → API → Backend Services → Domain → Database. Repositories are the only layer permitted to issue SQLAlchemy queries.

3. AI / Agent Layer

A parallel stack handles reasoning and action. An Orchestrator agent receives a goal, decides which specialized agents are relevant, and coordinates their responses. Agents never talk to the database directly — all reads and actions go through MCP tools, which ultimately call the same backend services and repositories the application uses.



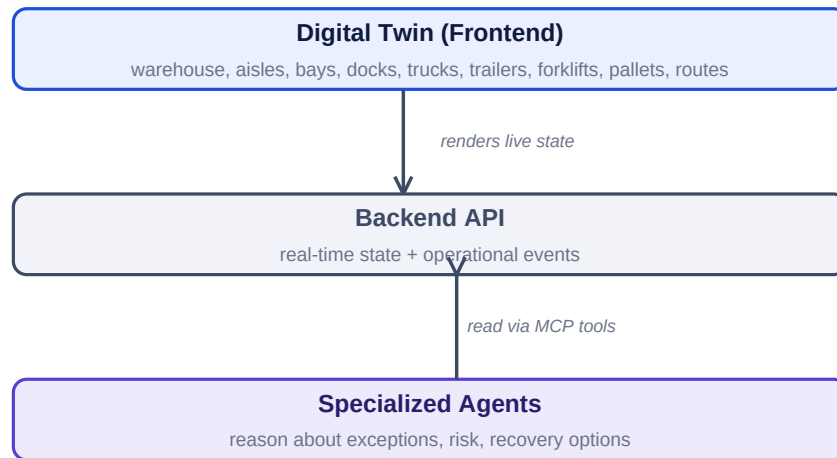
Same underlying PostgreSQL database as the application stack — the digital twin and the agents are two views of one source of truth, not two systems.

Specialized agents (each owns one operational domain)

- | | | | |
|-----------------|---------------------|--------------|----------------|
| • Communication | • Cost Optimization | • Demand | • Document |
| • Exception | • Inventory | • Logistics | • Orchestrator |
| • Simulation | • Supplier Risk | • Validation | • Warehouse |

4. Digital Twin ↔ Agent ↔ Data Relationship

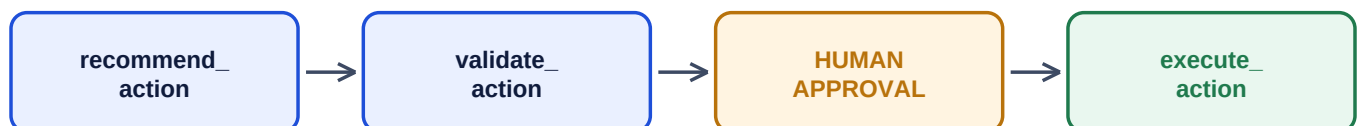
The digital twin is a **view** of the same operational data the agents reason over — never a separate source of truth. Actions agents take, once approved, flow back through the backend as **OperationalEvents**, which both the digital twin and the Control Tower UI subscribe to, closing the loop.



Recommendations → human approval → actions → operational events, fed back into both the digital twin and the backend API.

5. Human-in-the-Loop Principle

No agent-recommended action executes automatically. This is a hard architectural constraint, not just a UI affordance — the **validation** agent and the **execute_action** MCP tool are deliberately separate from **recommend_action** so a human approval gate always sits between them.



No agent-recommended action executes automatically — approval is a hard architectural gate, not a UI affordance.

6. Repository Structure

Folder	Purpose
agents/	Orchestrator + specialized agent definitions
backend/	FastAPI service (API → services → domain → repositories → DB)
data/	Local raw / processed / sample data
database/	Alembic migrations + seed data
docs/	Architecture, API, agent, MCP, A2A, and domain documentation
frontend/	React + TypeScript + Vite + Three.js digital twin UI
infra/	Docker / deployment configuration
mcp/	MCP server exposing operational tools to agents
scripts/	Setup, database, and dev helper scripts
tests/	Cross-cutting integration / e2e tests

7. Current Status & Roadmap

X	Repository foundation (this phase)	DONE
>	Backend domain models + first migrations (Warehouse, Dock, Truck, etc.)	NEXT
-	First MCP tools (read-only: `get_truck_status`, `get_inventory_position`, ...)	TODO
-	Orchestrator + first specialized agent (e.g. Exception agent) wired to MCP	TODO
-	Frontend Control Tower shell (layout, routing, live data binding)	TODO
-	Digital Twin scene v1 (static warehouse + trucks, no interactions)	TODO
-	Simulation + recommendation + human approval + action execution loop	TODO
-	A2A communication layer between agents	TODO

8. Recent Changes (Last 7 Days)

2 commit(s) touching 190 file(s) since last week's run.

2026-08-21 d526662 chore: clean up build/lint artifacts accidentally staged during verification

2026-08-21 4070037 chore: initialize ChainPilot architecture

Source: README.md, ARCHITECTURE.md, and git history, ChainPilot repository. This document is regenerated automatically every week.